

《A Survey of On-Policy Distillation for Large Language Models》，Mingyang Song 和 Mao Zheng 的 OPD 综述，共 89 页。它不是提出单一算法的论文，而是试图给整个 **On-Policy Distillation** 建立统一框架。论文把 OPD 归纳成三个核心设计轴：**优化什么 (Objective)**、**监督信号从哪来 (Signal)**、**训练怎么稳定/提效 (Dynamics)**。我按论文的章节顺序讲，重点放在“这一章到底解决什么问题、关键公式/方法是什么、和实际训练有什么关系”。

1. Introduction: 为什么需要 On-Policy Distillation?

第一章其实围绕一个问题：

传统 Knowledge Distillation 到底哪里有问题？ 传统 KD 基本是：

$$(x, y) \sim D_{\text{teacher}}$$

Teacher 先生成一批正确数据：



Student 永远在 Teacher 的“正确 prefix”上训练。但真正推理时：



Prompt

↓

Student

↓

$y_1 \rightarrow y_2 \rightarrow x_3 \rightarrow ???$

↑

自己犯错

一旦第三步错了，后面的状态：

$$(x, y_1, y_2, x_3)$$

可能训练时根本没见过。这就是 **Exposure Bias**。论文引用 DAgger 的经典分析：

$$\text{Off-policy error} = O(\epsilon T^2)$$

而理想的 on-policy interactive imitation 可以降到：

$$O(\epsilon T)$$

因此 OPD 的关键变化不是简单地：

“换一种 loss。” 而是：**改变训练数据来自哪里**。Student 自己 rollout：

$$y \sim p_\theta(\cdot|x)$$

然后 Teacher 对 **Student 实际访问到的状态**提供监督。这也是整篇论文最重要的定义。

2. Background and Unified Math: OPD 的数学基础

这是整篇论文理论上最重要的一章。

2.1 Classical Knowledge Distillation

先从经典 KD 开始。Teacher logits：

$$z_T$$

经过 temperature:

$$p_T^{(\tau)}(y|x) = \frac{\exp(z_y/\tau)}{\sum_j \exp(z_j/\tau)}$$

Student 学 Teacher:

$$D_{KL}(p_T || p_\theta)$$

对于 LLM, 就是逐 token:

$$L_{\text{Token-KD}} = \mathbb{E}_{x, y \sim D} \left[\sum_t D_{KL}(p_T(\cdot|x, y_{<t}) || p_\theta(\cdot|x, y_{<t})) \right]$$

问题就在:

$$y_{<t} \sim D$$

这个 prefix 来自数据集, 而不是 Student。论文强调, 经典 token KD 虽然计算方便, 但默认了一个“无错误历史”。Sequence KD 也没有真正解决。它只是:

$$\hat{y} = \arg \max_y P_T(y|x)$$

然后:

$$L = -\log P_\theta(\hat{y}|x)$$

仍然是:

Teacher rollout → Student 学习。所以还是 off-policy。

2.2 Off-Policy Exposure Bias

这一节正式解释为什么:

reasoning 越长, OPD 越重要。假设每一步 Student 有:

$$\epsilon$$

概率偏离 Teacher。如果每一步独立, 10 步 reasoning, 每步正确率 95%:

$$0.95^{10} \approx 60\%$$

已经掉得非常厉害。实际情况更糟, 因为:

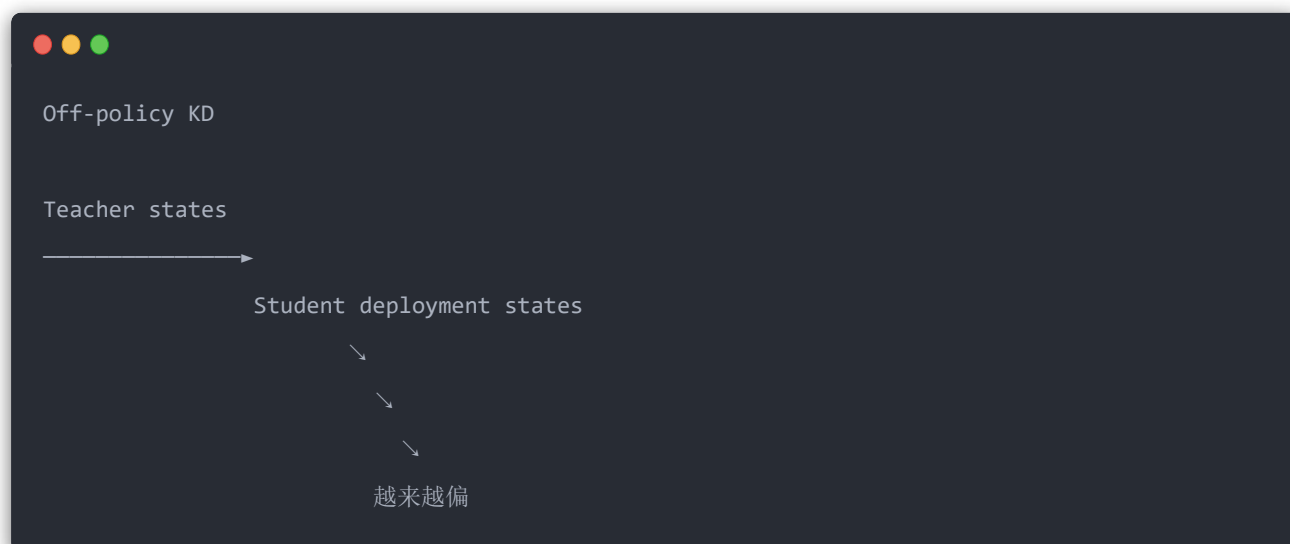
前一步错误会改变后一步输入分布。所以不是简单:

$$O(\epsilon T)$$

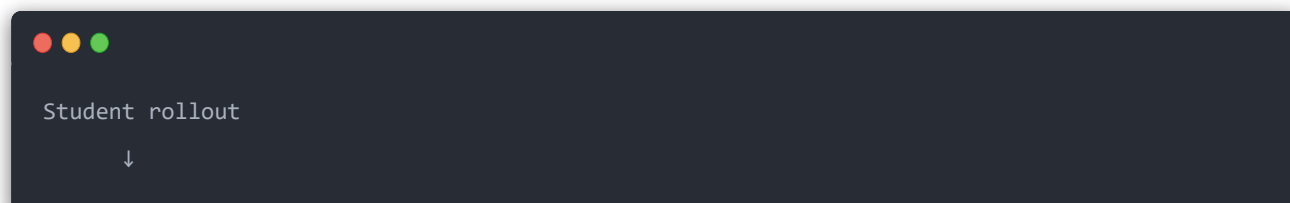
而可能出现:

$$O(\epsilon T^2)$$

的 compounding error。论文特别指出, 这对数学证明、代码等长链推理任务尤其严重。因此:



OPD 则:



Student 实际访问的 state
↓
Teacher 在这些 state 上指导
↓
Student update
↓
重新 rollout

2.3 Unified f-Divergence Framework

这是论文最核心的统一公式：

$$L_{\text{OPD}}(\theta) = \mathbb{E}_{y \sim \pi_{\text{mix}}} \left[\sum_t D_f(p_T(\cdot | x, y_{<t}), p_\theta(\cdot | x, y_{<t})) \right]$$

这个公式实际上把 OPD 拆成两个独立问题：

第一：trajectory 从哪里来？

$$y \sim \pi_{\text{mix}}$$

例如 GKD：

$$\pi_{\text{mix}} = \lambda p_\theta + (1 - \lambda) p_{\text{data}}$$

于是：

- $\lambda = 0$ ：传统 off-policy KD
- $\lambda = 1$ ：纯 OPD
- $0 < \lambda < 1$ ：混合

第二：Student 怎么学 Teacher？

就是：

$$D_f(p_T, p_\theta)$$

论文重点比较四种 divergence:

Loss	特性	更适合
Forward KL	Mode-covering	开放生成
Reverse KL	Mode-seeking	数学/代码
JSD	两者折中	翻译等
α -divergence	可调节	自适应任务

Reverse KL:

$$D_{KL}(p_{\theta}||p_T)$$

倾向于:

Teacher 有 5 种解法，我找其中一种最靠谱的学。 Forward KL:

$$D_{KL}(p_T||p_{\theta})$$

更倾向:

Teacher 的所有模式我都尽量覆盖。 所以数学、代码这种 “一条正确路线就够” 的任务，论文认为 RKL 的 mode-seeking 性质往往更合适；开放生成则更需要 FKL 的 mode-covering。

2.4 Distillation Scaling Laws

这一节问一个非常实际的问题:

Teacher 是不是越大越好？ 答案: **不是**。 存在 Teacher–Student capacity gap。 例如:

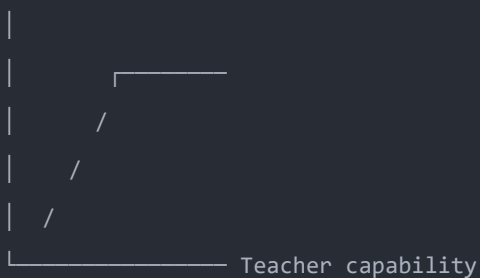


Teacher

7B → 1.5B ✓
14B → 1.5B ✓
70B → 1.5B ?
671B ▶ 1.5B ???

Teacher 太强后，它的 probability distribution 可能已经超出 Student 能表达的范围。因此性能可能：

Student quality



先增长，再饱和。论文指出 OPD 还有传统 KD 没有的 scaling dimension:

$R = \text{rollout budget}$

未来真正需要研究的是：

$$Quality = f(N_T, N_S, D, R)$$

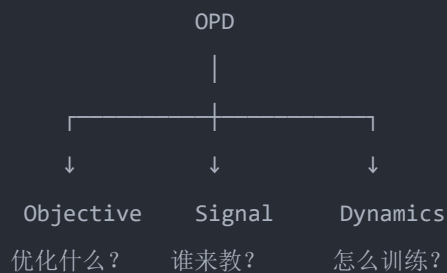
也就是 Teacher 大小、Student 大小、数据量、rollout budget 如何联合 scaling。

3. Landscape and Method Selection: OPD 方法到底怎么分类？

第三章是这篇综述的“地图”。论文提出三个轴：

$$OPD = Objective + Signal + Dynamics$$

也就是：



论文第 9 页的 Figure 1 就是整篇文章最值得保存的一张图：把大量 OPD 方法按照这三个设计轴组织起来。其中 Signal 又分：

```
graph TD; Signal --> ET[External Teacher]; Signal --> SD[Self-Distillation]; ET --> WB[White-box]; ET --> BB[Black-box];
```

这是后面第 4、5、6 章的总纲。

4. Objective Functions: 到底优化什么?

第 4 章回答：

Student rollout 已经有了，Teacher 也有了，现在 loss 怎么设计？论文将其分成三代：

Fixed $\rightarrow$ *Adaptive* $\rightarrow$ *RL-Augmented*

4.1 Fixed Divergence

第一代就是：

$$D_{KL}(p_T \| p_S)$$

或者：

$$D_{KL}(p_S \| p_T)$$

或者 JSD。代表：

- GKD
- MiniLLM
- DistiLLM 其中 GKD 是非常重要的 baseline:

$$\pi_{\text{mix}} = \lambda p_S + (1 - \lambda) p_{\text{data}}$$

MiniLLM 则重点使用 Reverse KL:

$$D_{KL}(p_S \| p_T)$$

并通过 REINFORCE 优化 sequence-level objective。

4.2 Adaptive Divergence

固定 KL 有一个明显问题：

所有 token 都用同一种力度学 Teacher。但实际上：



token A:

Teacher 很确定

Student 很确定

→ 很值得学

token B:

Teacher 很不确定

Student 也很不确定

→ 不应该强行学

token C:

Teacher/Student 严重冲突

→ 可能是 Student 已经进入错误 prefix

所以开始出现：

- EOPD
- AOPD
- Position-Weighted OPSD
- Token-Teachability
- Trust-Region OPD 本质是：

$$L_t = w_t D(p_T, p_S)$$

其中：

$$w_t = f(H_T, H_S, D(p_T, p_S), position, confidence, \dots)$$

也就是不同 token 的 Teacher supervision 权重不同。

4.3 RL-Augmented OPD

这一部分和你前面问的 RL 联系最紧。 核心问题：

如果 Student 永远模仿 Teacher，理论上怎么超过 Teacher？ 纯 KD：

$$Student \rightarrow Teacher$$

最终 ceiling 就是 Teacher。 所以加入：

$$Reward$$

变成：

$$L = L_{\text{distill}} + \lambda L_{\text{RL}}$$

这就出现：

- G-OPD
- RLKD

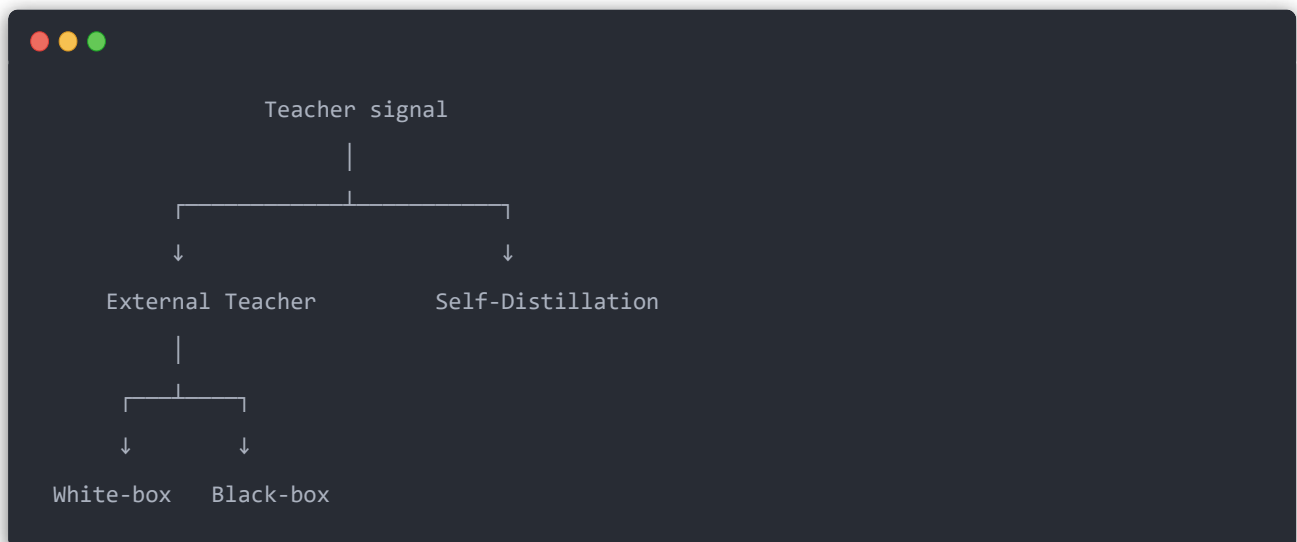
- KDRL
- CoDistill-GRPO
- dGRPO
- TGPO 等。论文特别强调一个理论联系：

$$OPD \approx KL\text{-constrained RL}$$

所以现在 OPD 和 RL 的边界其实已经开始模糊。论文也指出，两者都使用 on-policy rollout、trajectory signal 和 policy update，核心差异主要变成了**监督信号来自 Teacher distribution 还是 reward**。

5. Signal Source: 到底谁来当 Teacher?

这一章我认为是全文第二重要的一章。论文把 Teacher 分成：



5.1 White-box Teacher

你能拿到：

$$z_T \in R^{|V|}$$

也就是 Teacher 完整 logits。这样就可以真正计算：

$$D_{KL}(p_T || p_S)$$

这是信息量最大的 OPD。但问题也最大：

例如：

$$[B, T, V]$$

假设：

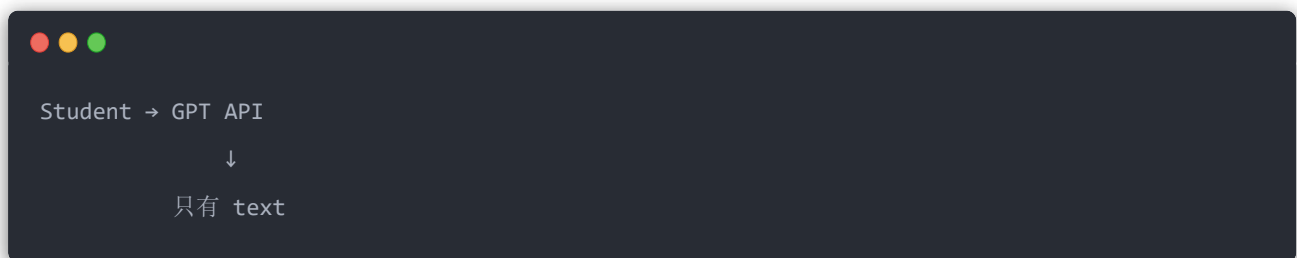
- $B=16$
- $T=4096$
- $V=128K$
- BF16 一个 batch 的 logits 大约：

$$16GB$$

论文明确指出，这在单机 NVLink 环境还可以处理，但 multi-node 就需要 tensor slicing 或 top-k sparsification。

5.1.2 Black-box

如果 Teacher 是 API：



拿不到 logits：

$$p_T(v|x)$$

那怎么办？ 只能使用：

- Teacher text
- score
- preference
- critique

- verbal feedback 代表:
- Lion
- GAD
- OVD
- LUFFY
- ROPD 这时 OPD 开始越来越像 RL。

5.2 Self-Distillation

这是 2025–2026 增长最快的一类。 最有趣的问题:

没有更强 Teacher, 自己怎么教自己? 答案是制造 **information asymmetry**。 例如 Student:

$$p_{\theta}(y|x)$$

Teacher 其实还是同一个模型, 但多给它答案:

$$p_{\theta}(y|x, \boxed{GT})$$

于是:

```
Student:
Question
  ↓
Model

Self-Teacher:
Question + Ground Truth
  ↓
Same Model
```

因为 Teacher 看到了 privileged information, 所以:

$$p_T \neq p_S$$

然后：

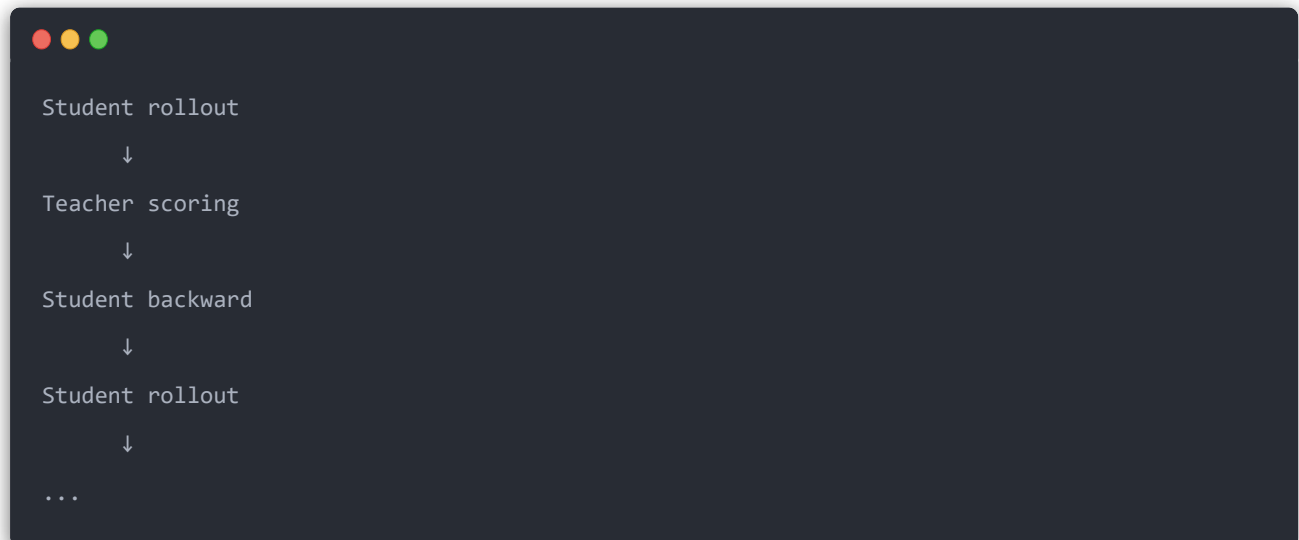
$$D_{KL}(p_S \| p_T)$$

这就是 OPSD 一类方法的核心。因此：

Self-distillation 不是简单让模型复制自己，而是制造一个“知道更多信息的自己”。

6. Training Efficiency & Stabilization

这一章最工程化。因为 OPD 的真实训练 loop：



数据分布一直变。所以比 SFT 难稳定很多。论文将问题概括为 non-stationary rollouts、旧 rollout 过期以及 hard prompt 下梯度 SNR 下降等。

6.1 Token / Sample Weighting

核心思想：

不要所有 token 都蒸馏。例如：

$$L = \sum_t w_t D_t$$

只重点学习：

- Teacher 高 confidence
 - Teacher/Student divergence 高
 - 高信息 token
 - reasoning关键位置 代表 TIP、SCOPE、SelectKD 等。这和 RL 的 token credit assignment 已经非常接近。
-

6.2 Curriculum

不要：

```
Day 1:
超难 AIME
Student 完全不会
Teacher 强行指导
```

而是：

```
Easy
↓
Medium
↓
Hard
↓
Very Hard
```

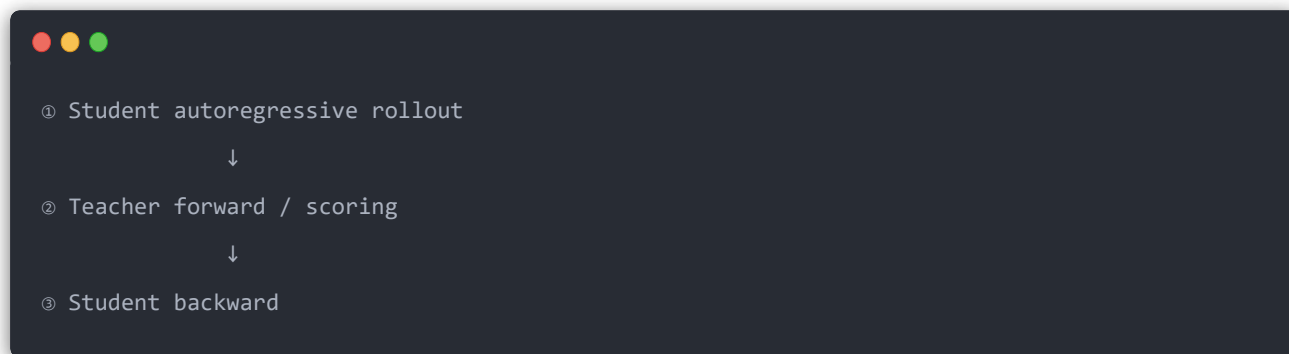
让 Student distribution 慢慢靠近 Teacher。因此：

$$Difficulty(t) \uparrow$$

代表 PACED、Stable-OPD、CaOPD 等。

6.3 Compute Optimization

这是做大模型系统的人最值得看的部分。标准 OPD 有三个主要计算阶段：



其中 Teacher scoring 对大 Teacher 尤其昂贵。所以出现三个非常典型的方向。

FOPD: 少 rollout

发现前面 token 信息量高，所以只做：



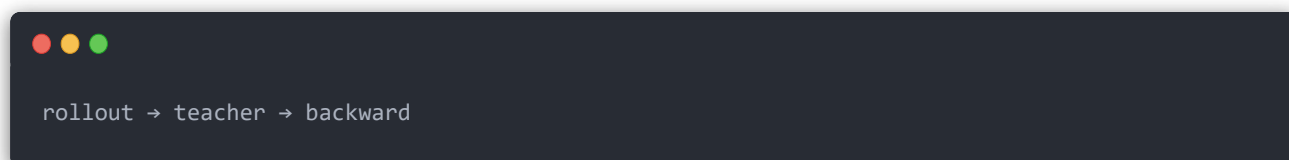
论文报告在相应实验设置下可以减少约 **2×–47× FLOPs**。

Lightning-OPD: 缓存 Teacher

尽量避免每次都重新 Teacher inference。

NPD: 异步

把：



改成流水：

```
GPU group A: rollout _____
GPU group B:  teacher _____
GPU group C:   training _____
```

论文报告 NPD 相对同步 baseline 有 **$8.1\times$ throughput speedup**。此外 Prune-OPD 会根据 Student/Teacher 的 top-k overlap 动态检测 prefix drift；一旦 Teacher 对当前轨迹的指导开始不可靠，就降权甚至提前截断，论文汇总的实验中训练时间减少 37.6%–68.0%。

7. Understanding OPD: OPD 为什么有效，又为什么会炸？

第 7 章其实是全文最值得精读的一章。它回答四个问题：

When does OPD work?

Why does OPD fail?

Why mathematically?

When is OPD worth the cost?

7.1 Success Conditions

一个非常重要的结论：

Teacher 强，并不意味着一定能教好 Student。Student 和 Teacher 首先需要有：

compatible reasoning patterns

论文用：

Top-K overlap

来衡量。例如：



Teacher top-k:
[therefore, x, equals, 3...]

Student top-k:
[therefore, x, equals, 4...]

overlap 高
→ 可以 OPD

但:



Teacher:
[Let, theorem, suppose...]

Student:
[答案, 是, ...]

overlap 极低

这时候直接 OPD 可能不好。应该先:

$$SFT/Off - policy\ warmup \rightarrow OPD$$

论文明确把 reasoning-pattern compatibility 视作有效 OPD 的必要条件之一。

7.2 Failure Modes

这里列了很多非常重要的失败模式。其中我认为最值得记住的是:

Flawed Prefix Trap

Student:



A → B → WRONG → X

然后你问 Teacher:

在 `A B WRONG X` 后面应该生成什么? 问题是 Teacher 自己可能从来没访问过这种 state。于是:

$$p_T(\cdot | A, B, WRONG, X)$$

本身也可能不可靠。这意味着:

On-policy 并不是 Student rollout 越多越好。 Student 偏离 Teacher 太远以后, Teacher supervision 可能反而变成噪声。这也是前面:

- token weighting
- trust region
- Prune-OPD
- curriculum 存在的重要原因。

7.3 Unified Theory

这里重新统一:

- f-divergence
- imitation learning
- RL
- KL-constrained optimization 一个非常有意思的视角是:

$$Reward_t \approx \log p_T(y_t | s_t) - \log p_S(y_t | s_t)$$

也就是说 Teacher probability 本身可以理解成一种:

dense token reward。 于是:



RLVR

token token token token token

↓

final reward

而 OPD:



token token token token token

↓

↓

↓

↓

↓

r1

r2

r3

r4

r5

这就是 OPD 和 RL 最深层的联系之一。

7.4 OPD vs Off-policy: 到底什么时候值得用?

论文没有说:

OPD 一定比 SFT 好。相反它给了比较务实的判断。如果:

- Teacher 极强;
- Student 很小;
- reasoning 不长;
- 静态 Teacher 数据覆盖充分; 那么:

Off-policy SFT

可能已经足够。论文甚至专门讨论 DeepSeek-R1 distillation: R1 的强 Teacher + 高质量多样 CoT, 让纯 off-policy distillation 依然可以非常强。但对于:

- 长 reasoning
- code
- math
- agent

- 强 Student static dataset 越来越覆盖不了 Student 实际 trajectory。这时 OPD 的收益更大。工程上论文推荐的思想其实很像：

Cheap Off-Policy $\rightarrow$ *Expensive On-Policy*

也就是先用便宜数据把 Student 拉近，再把昂贵 OPD compute 留给最后的质量提升。

8. Applications & Systems

第八章从算法转到真实系统。涉及：

- Qwen
- DeepSeek
- Agent
- Code
- Multimodal
- GUI Agent
- Robotics
- Healthcare
- Speculative decoding 其中一个很重要的趋势是：

OPD 的粒度正在从 token 扩展到 step、turn、skill、trajectory。例如 Agent：

```
Trajectory
|
├─ Turn 1
|   ├── reasoning
|   └─ tool call
|
├─ Turn 2
|   ├── observation
|   └─ reasoning
|
└─ Turn 3
```

如果只给最终：

$$Reward = 0$$

credit assignment 太困难。所以开始出现：

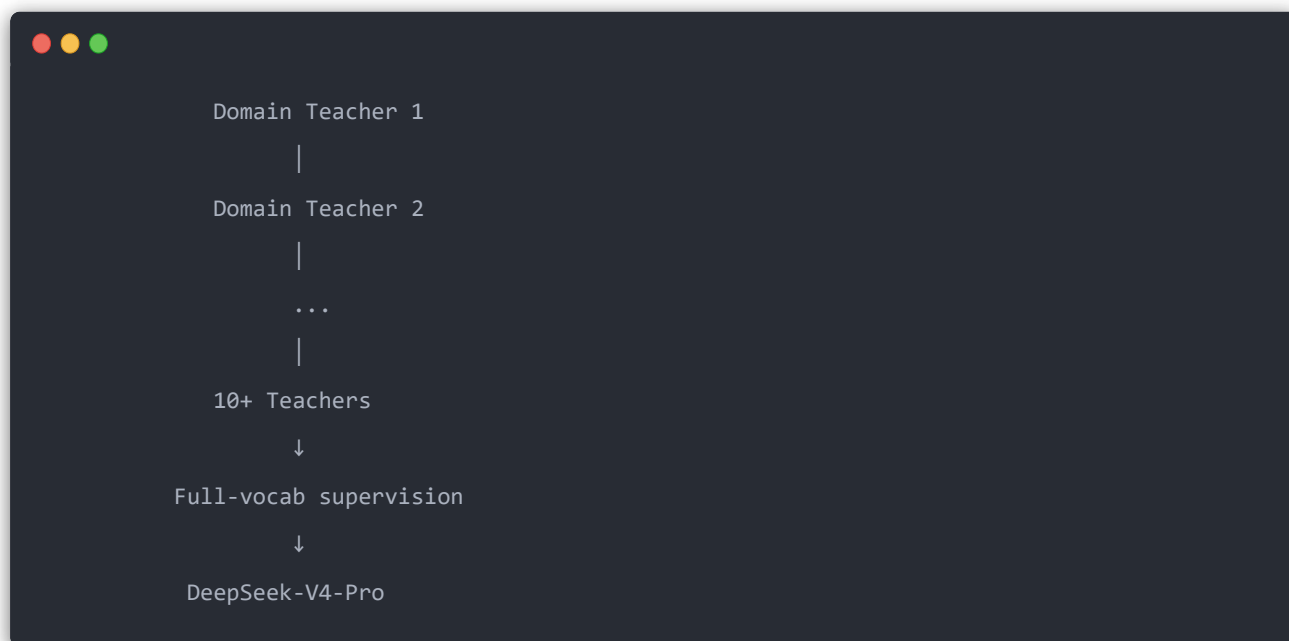
$$token \rightarrow step \rightarrow turn \rightarrow skill \rightarrow trajectory$$

论文总结出一个很漂亮的原则：

Distillation granularity 应该与 error compounding 的 granularity 匹配。

8.x DeepSeek-V4：我建议你重点看

因为你最近关注 DeepSeek-V4，这篇 survey 里 DeepSeek-V4 是一个很有价值的工业 OPD 案例。论文描述的是：



真正麻烦的是：

$$10 + Teachers \times 100K + vocabulary \times sequence \times batch$$

如果直接保存：

$$[B, T, V]$$

logits, 开销巨大。论文描述 DeepSeek-V4 的工程思路是缓存 **last-layer hidden states**, 而不是简单把所有 Teacher 的完整 vocabulary logits 全部物化保存。这一块其实很值得单独拆出来看, 因为它已经不是 “OPD 算法”, 而是:

如何真正把 multi-teacher OPD 跑到 trillion-parameter scale。

9. Open Problems

最后一章讨论未来问题。第一个就是前面提到的:

OPD Scaling Law

现在我们知道:

$$\text{Pretraining} : \text{Quality} = f(N, D, C)$$

但是 OPD:

$$\text{Quality} = f(N_T, N_S, D, R, \text{Teacher inference}, \text{Student rollout})$$

目前还没有完整理论。论文指出, 现有 distillation scaling law 更多还是 off-policy, 而 OPD 多出来一个非常关键的 Student rollout compute axis。

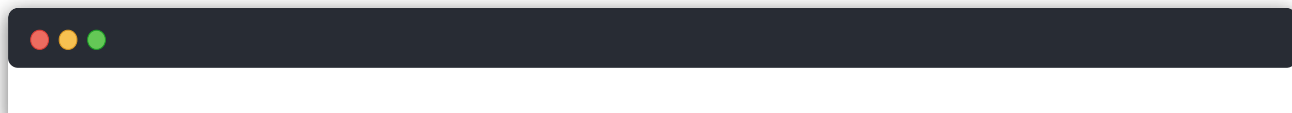
另一个我认为非常重要:

Teacher uncertainty

现在 OPD 默认:

$$\text{Teacher} = \text{correct}$$

其实并不成立。尤其:



```
Student bad prefix
  ↓
Teacher sees OOD state
  ↓
Teacher uncertain
  ↓
仍然强制 KL
  ↓
错误 supervision
```

未来应该使用：

$$w_t = f(\text{Teacher uncertainty})$$

例如 Teacher entropy 很高：

$$H(p_T) \uparrow$$

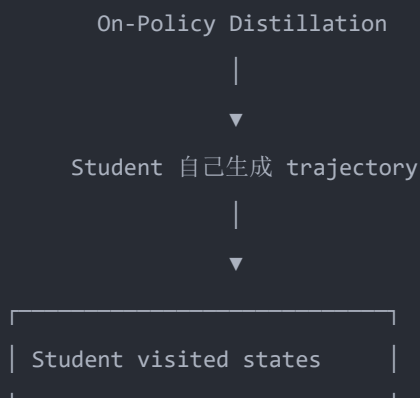
就让：

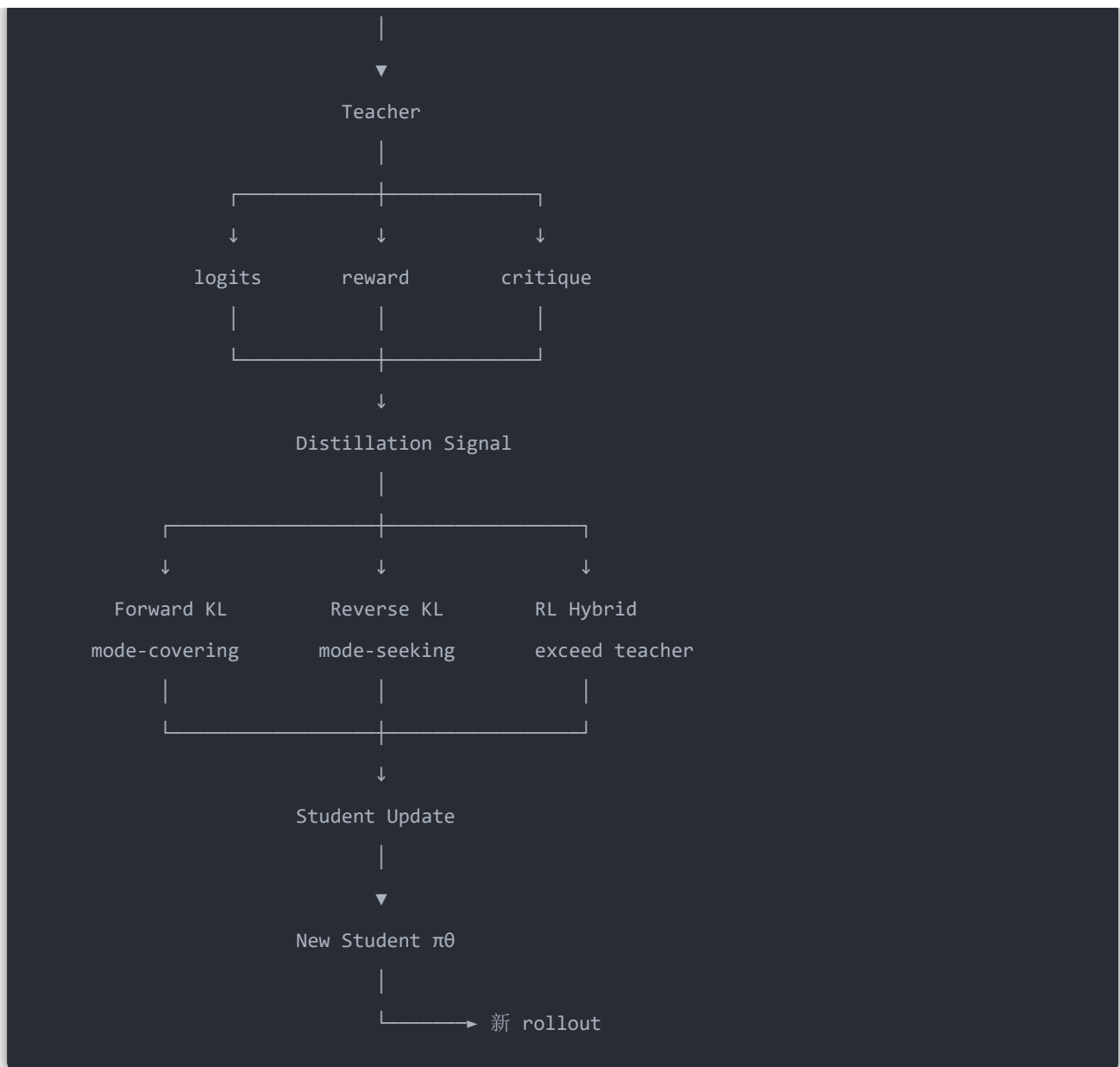
$$w_t \downarrow$$

论文明确把这个问题和 flawed-prefix trap 联系起来：当 Teacher 在 Student 的异常 prefix 上高度不确定时，应抑制对应位置的 distillation gradient。

把整篇论文压缩成一张逻辑图

你可以这样记：





所以整篇 89 页最终其实是在回答三个核心问题：

What to optimize?

第 4 章：FKL / RKL / Adaptive / RL。

Who teaches?

第 5 章：White-box / Black-box / Self-distillation。

How to make it work?

第 6–8 章：weighting、curriculum、异步、截断、multi-teacher、Agent，以及失败模式。而第 2、7 章给它们提供理论解释。如果你的目标是理解最新 reasoning model 的 post-training，我建议精读顺序不是 1→9，而是：

2.3 → 4.1 → 4.3 → 7.3 → 7.2 → 6 → 8

也就是先搞明白 **f-divergence** → **RKL/FKL** → **OPD 与 RL 的等价关系** → **为什么会失败**
→ **怎么工程化**。这条线比按页顺序读更容易把 OPD 真正串起来。